Reviewer Pack — Minimal Rank of Category Morphisms vs Monotone Circuit Lower…

Entry 3986fd2f9e4c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 05:40:30 UTC

Minimal Rank of Category Morphisms vs Monotone Circuit Lower Bounds for k-CLIQUE

Entry ID: `3986fd2f9e4c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 05:40:30 UTC

1. Conjecture

Field A (mathematical branch): Category Theory (Morphisms) **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

> ['For any given graph G = (V, E) that is a k-Clique, the minimal rank of the category morphism from the free monoidal category generated by the vertices to the category of distributive lattices is upper-bounded by a function f(n) = O(n^k), where n is the number of vertices in G.', 'This upper bound holds for all k-CLIQUE instances with n ≤ 40, and the ratio of minimal rank to n^k approaches a constant value as n grows.', "For any counterexample instance G', if there exists no category morphism from the free monoidal category generated by V(G') to the category of distributive lattices with rank less than f(n), then G' is not a k-CLIQUE graph."]

Rationale (proposer's reasoning):

> ['Category theory provides a powerful framework for studying algebraic structures, and its use in complexity theory has been limited. This conjecture proposes a connection between the minimal rank of category morphisms and the structure of k-CLIQUE graphs, which are known to be hard for monotone circuits.', 'If this conjecture is true, it would demonstrate that category-theoretic tools can capture the computational hardness of k-CLIQUE problems, providing new insights into monotone circuit complexity.', 'Furthermore, the use of rank as an invariant in this context may reveal hidden structure within the problem that could be exploited for algorithm design.']

Taxonomy category: `Category Theory × Monotone Circuit Complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ad5bb5796929f30d`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all k-CLIQUE graphs G with n ≤ 40 vertices, the ratio of the minimal rank of a category morphism to n^k is less than or equal to f(n), where f(n) = O(n^k). The conjecture is falsified if any graph G' with n ≤ 40 vertices has no category morphism from its free monoidal category to the category of distributive lattices with rank less than f(n).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "category morphisms" AND "k-Clique" AND minimal rank" - "free monoidal category" AND distributive lattices" AND monotone circuit lower bounds" - "upper bound" ON "minimal rank of category morphisms" FOR k-CLIQUE graphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_k_clique(G, k):
        n = len(G)
        if n < k:
            return False
        for i in range(n):
            for j in range(i+1, n):
                if (i, j) not in G and (j, i) not in G:
                    return False
        return True

    def free_monoidal_category(V):
        n = len(V)
        category = {}
        for v in V:
            category[v] = {v}
        for v1 in V:
            for v2 in V:
```

```
            if v1 != v2:
                category[(v1, v2)] = {v1, v2}
    return category

def morphism_space(C, D):
    n = len(C)
    m = len(D)
    space = []
    for i in range(n):
        for j in range(m):
            if C[i] <= D[j]:
                space.append((i, j))
    return space

def min_rank(M):
    n = len(M)
    rank = 0
    while M:
        row = next(iter(M))
        rank += 1
        M -= {row}
        for r in list(M):
            if any(x in r for x in row):
                M.remove(r)
    return rank

def f(n, k):
    return n**k

n = random.randint(5, 40)
G = set()
while len(G) < n:
    u, v = random.sample(range(n), 2)
    if (u, v) not in G and (v, u) not in G:
        G.add((u, v))

if not is_k_clique(G, k):
    return {
        "metric_name": "min_rank_to_nk_ratio",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "not_a_k_clique"
    }

V = list(G)
C = free_monoidal_category(V)
M = morphism_space(C, {frozenset([v]) for v in V})
rank = min_rank(M)

ratio = rank / f(n, k)

return {
    "metric_name": "min_rank_to_nk_ratio",
    "metric_value": ratio,
    "instances_tested": 1,
```

```
        "conjecture_holds": ratio <= 1.0,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    total_ratio = sum(r["metric_value"] for r in results if r["metric_value"] is not None)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["metric_value"] is not None for r in results):
        print(f"RESULT: SUPPORTED mean={total_ratio/len(results)} std=0.0 support_fraction={suppor
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(s for s, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not_a_k_clique\" first_failing_seed={first_fail
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c2963d47.py", line 104, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c2963d47.py", line 74, in run_trial
    if not is_k_clique(G, k):
                          ^
NameError: name 'k' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed due to an undefined variable ‘k’, which prevents us from verifying whether the conjecture’s pre-registered support condition has been met. | next: Define the variable ‘k’ in the code and rerun the test to verify the conjecture’s conditions.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15543
2	propose	ollama_remote	glm4:latest	0	0	15105
3	preregistration	ollama_remote	glm4:latest	0	0	10205
4	novelty	ollama_remote	glm4:latest	0	0	8423
5	novelty	ollama_remote	glm4:latest	0	0	8899
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20042
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9368
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9649
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9749
10	judge	ollama_remote	glm4:latest	0	0	14710

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 121694 ms total latency. Provider mix: {‘ollama_remote’: 10}

(full prompt+response transcripts available in `research/audit/3986fd2f9e4c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3986fd2f9e4c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3986fd2f9e4c.tar.gz` (if generated)